CS/SE 4348-505: Operating Systems Concepts
Project 2

Instructor: Mingming Chen
Assigned on: Monday, October 12, 2026
Due date: Sunday, November 1, 2026

This is a team assignment. Each team should consist of two students. You must register your team using the sign-up method announced on Canvas by **Saturday, October 17, 2026 at 11:59 p.m**. Students who have not joined a team by that deadline will be assigned to teams randomly by the instructor. Team changes after the deadline require prior approval.

Each team is expected to develop and understand its own solution. You may discuss general implementation ideas with other students, but you may not copy another team's code or submit work that you do not understand.

Students may use generative AI tools only in ways permitted by the course syllabus. In particular, AI tools may be used to help understand concepts, explain error messages, or obtain general debugging guidance, but may not be used to generate substantial portions of code or complete solutions. All submitted work must reflect the students' own understanding and effort.

Each team is required to complete a demonstration of its project to the TAs; otherwise, the team will not receive any credit. Both team members must be present and must independently demonstrate their understanding of the submitted work. Detailed demonstration requirements are provided in Section 4.

1 Project Description

Important: The goal of this project is to practice multithreaded programming in C++, mutual exclusion using atomic shared variables, and the design tradeoffs between coarse-grained and fine-grained synchronization.

You will implement two solutions to the Dining Philosophers problem:

1. a **coarse-grained solution**, in which one lock protects all chopsticks; and
2. a **fine-grained solution**, in which each chopstick is protected by a separate lock.

You may not use the built-in lock or mutex classes provided by C++ or the operating system. Instead, you must build the required locks using Peterson's two-thread mutual-exclusion algorithm.

Part 1: Peterson Two-Thread Lock

Implement a lock for exactly two participating threads using Peterson's mutual-exclusion algorithm. Each participant must use a distinct local identifier, 0 or 1, when acquiring the lock. All shared variables used by the Peterson lock must be declared using `std::atomic` from the standard `atomic` header:

https://en.cppreference.com/w/cpp/header/atomic

Your lock should provide clearly defined acquire and release operations. You may choose appropriate method names, but their purpose and parameters must be documented.

Part 2: Tournament-Tree Lock

Using your two-thread Peterson lock as the building block, implement an n-thread lock using a Tournament-Tree (TT) design. Each internal tree node must be an instance of your Peterson lock. A thread acquires the locks along its path from its leaf toward the root and releases them in the reverse order, from the root toward its leaf.

Your TT lock must support every $n \geq 2$; n is not necessarily a power of two. Your design must explain how threads are assigned to leaves and how unused positions are handled when n is not a power of two. An array or vector representation of the tree is recommended. Writing pseudocode for the TT lock before implementing it is also strongly recommended.

Part 3: Dining Philosophers

Philosophers and chopsticks are numbered from 0 to $n - 1$. Philosopher i is represented by thread i. Chopstick i is located between philosophers i and $(i + 1) \bmod n$.

For the **coarse-grained solution**, use one n-thread Tournament-Tree lock to protect access to all chopsticks. A philosopher must acquire this lock before picking up its two chopsticks and release it after putting them down.

For the **fine-grained solution**, protect each chopstick with a separate two-thread Peterson lock. Only the two philosophers adjacent to a chopstick may participate in that chopstick's lock. Your solution must prevent neighboring philosophers from eating simultaneously and must be deadlock-free. Explain your deadlock-prevention strategy in the `README`.

Each philosopher must complete **five** thinking/eating cycles and then terminate. Simulate each invocation of `thinking()` and `eating()` using a new random delay from 1 to 500 milliseconds.

Required Output Format

Because thread schedules are nondeterministic, the exact order of output lines will vary. However, every output line must follow the format below so that the execution can be checked automatically:

```
BEGIN COARSE
COARSE 2 0 THINK_START
COARSE 2 0 THINK_END
COARSE 2 0 EAT_START
COARSE 2 0 EAT_END
...
END COARSE
BEGIN FINE
FINE 2 0 THINK_START
FINE 2 0 THINK_END
FINE 2 0 EAT_START
FINE 2 0 EAT_END
...
END FINE
```

Each activity line has four whitespace-separated fields:

`solution philosopher_id round event`

where `solution` is `COARSE` or `FINE`, `philosopher_id` is in the range 0 to $n-1$, `round` is in the range 0 to 4, and `event` is one of `THINK_START`, `THINK_END`, `EAT_START`, or `EAT_END`.

Construct each event record completely before printing it, and emit the entire record using one output operation. For example, a complete record may be printed using one printf() call. Do not use std::mutex or another prohibited synchronization primitive solely to format the output. Do not write prompts, labels other than the required `BEGIN`/`END` lines, debugging messages, or other text to standard output. Error messages must be written to standard error.

The random-number generation used by the threads must not introduce a data race. Useful C++ references include:

https://en.cppreference.com/w/cpp/thread/sleep_for
https://en.cppreference.com/w/cpp/numeric/random

Program Invocation and Required Behavior

Write one C++ program and name its executable `my-dining`. It must accept one command-line argument:

```
./my-dining n
```

where $n \geq 2$ is the number of philosophers.

For each invocation, the program must run the coarse-grained solution first and the fine-grained solution second. Clearly label the output of each solution. For each solution, the main thread must create n philosopher threads numbered 0 through $n-1$ and use `join()` to wait for all of them to terminate before proceeding or exiting.

The program must validate the command-line argument and handle important failures gracefully. At a minimum, it must detect an incorrect number of arguments, a non-integer value of n, $n < 2$, allocation failure, and failure to create a thread. An unrecoverable error must produce a meaningful message on standard error and a nonzero exit status. The program must not hang indefinitely or leave joinable threads unmanaged.

Programming Language and Compilation

You must implement this project in **C++17**. Note that the compiler version and the C++ language standard are different. For example, `g++ 11.4.0` is a version of the GNU C++ compiler; it is not "C++11" and it is not "g++17." The `g++ 11.4.0` compiler supports the C++17 language standard.

Your submitted `Makefile` must explicitly select C++17 and enable POSIX thread support using:

```
g++ -std=c++17 -pthread
```

The command `make` must produce an executable named `my-dining`. Code that requires a newer C++ standard or compiler-specific extensions is not permitted. Any significant deviation from the project requirements may result in a penalty.

2 Implementation Constraints

- You must use `std::thread` to represent philosophers.

- You must use `std::atomic` for all shared variables in Peterson's algorithm.
- You may not use `std::mutex`, `std::recursive_mutex`, semaphores, condition variables, built-in spin locks, or other ready-made synchronization primitives to implement either required solution.
- Both the two-thread Peterson lock and the n-thread Tournament-Tree lock must be implemented by your team.

3 Automated Evaluation

An automated test kit will be used to evaluate the required interface and observable correctness of the program. The tester will analyze the event trace rather than compare it with one fixed output sequence. Test cases will include different values of n, including values that are not powers of two, such as $n = 3$ and $n = 5$.

The test kit will check whether:

1. `make clean` and `make` complete successfully and produce `my-dining`;
2. invalid command-line arguments are rejected with a nonzero exit status;
3. the program terminates within the allowed time for test cases such as $n = 2, 3, 5$, and 8;
4. every philosopher completes exactly five correctly ordered thinking/eating cycles in each solution;
5. no two philosophers eat simultaneously in the coarse-grained solution;
6. no two neighboring philosophers eat simultaneously in the fine-grained solution; and
7. the submitted source does not use prohibited ready-made synchronization primitives.

Passing the automated tests is necessary but not sufficient for full credit. The test kit cannot by itself prove that the required Peterson and Tournament-Tree algorithms were implemented correctly. Lock design, source-code quality, and individual understanding will also be evaluated through source inspection and the demonstration.

4 Demonstration Requirements

Each team will have approximately **10 minutes** to demonstrate its project. A prepared presentation is not required. During the demonstration:

1. The team must compile the submitted program using its submitted `Makefile`.
2. The team must run the program using test cases selected by the instructor or TA, including a value of n that is not a power of two.
3. Each team member must independently answer questions about Peterson's algorithm, atomic variables, the Tournament-Tree lock, deadlock prevention, coarse- versus fine-grained synchronization, and the submitted code.
4. The instructor or TA will determine which team member answers each question.
5. The team may be asked to predict program behavior, locate and explain specific code, or make a small modification.

6. AI tools, online assistance, communication with other students, and other external assistance may not be used during the demonstration.

The demonstration must use the version of the code submitted through Canvas. Each member is responsible for understanding the complete program, not only the portion that the student claims to have written. If a member cannot demonstrate reasonable understanding, that student may be required to complete an additional individual demonstration, and the student's individual grade may be adjusted.

5 Grading

This project is worth 10 points. The 4 implementation points are normally shared by the team. The remaining 6 points evaluate each student's individual understanding during the demonstration. Therefore, members of the same team may receive different project grades.

Category	**Points**
Correct implementations and required interface	2
Lock design, robustness, and submission quality	2
Individual demonstration questions	6
Total	**10**

For the 2-point *Correct implementations and required interface* category, the automated-test results will normally be interpreted as follows:

- **2 points:** all core automated tests pass;
- **1 point:** the program compiles and at least one required solution works substantially correctly, but the other solution or an important required case fails; and
- **0 points:** the program does not compile, neither required solution works, or the program consistently hangs.

Because AI assistance is permitted within the limits established by the course policy, the demonstration will focus primarily on each student's understanding of the submitted program. One team member may not answer for the other. A student who does not complete the required demonstration without an approved absence will not receive credit for the project.

6 Submission Information

Submit the project through Canvas. Each team member must submit the same team archive. Name the archive using the team number and both members' names, for example:

`Team1_MingmingChen_StudentName.zip`

The archive must contain:

1. all C++ source and header files;
2. a `Makefile` that produces the executable `my-dining`; and
3. a `README` containing the team number, both members' names, compilation and execution instructions, the fine-grained deadlock-prevention strategy, and any assumptions or known limitations.

Points will be deducted if required files are missing or if the submitted project cannot be compiled using the submitted `Makefile`.